Computer Organization and Architecture (CSL251)
LAB-6
2025-26 W

Assignment: RISC-V Privilege Modes, CSRs, and Trap Handling

Total Marks: 200 (4 × 50)
Submission: Assembly source code + required screenshots

Objective

The objective of this assignment is to understand the RISC-V privilege architecture, including Machine mode and User mode, and to gain hands-on experience with Control and Status Registers (CSRs), exceptions, interrupts, and trap handling mechanisms.

Problem 1: Switching Between Machine Mode and User Mode

(50 points)

Write a RISC-V assembly program that **continuously switches between Machine mode and User mode**.

Program Requirements

1. The program should start execution in **Machine mode**.
2. Modify the required CSRs (mstatus, mepc, and mtvec) to prepare for switching to User mode.
3. Use the mret instruction to transfer control from Machine mode to User mode.
4. The User mode code should be part of the same program but placed in a separate section or function.
5. In User mode, execute a few simple instructions and then use the ecall instruction to switch back to Machine mode.
6. Implement a **minimal machine-mode trap handler** to handle the ecall.
7. The trap handler should:
    - Read the values of mepc and mcause
    - Ensure control returns to the User mode code
8. The program should repeatedly switch between Machine mode and User mode.

Observations Required

- Values of mepc, mcause, mtvec, and mstatus in:
    - Machine mode
    - User mode
- Screenshots showing CSR values during mode transitions

Problem 2: Machine Timer Interrupt Handling

(50 points)

Write a RISC-V assembly program to demonstrate **machine timer interrupt handling**.

Program Requirements

1. Begin execution in **Machine mode**.
2. Initialize the mtvec register with the address of a machine-mode trap handler.
3. Enable machine-level interrupts by configuring:
    - mstatus
    - mie
4. Configure the machine timer to generate a **timer interrupt**.
5. When the timer interrupt occurs:
    - Control should jump to the trap handler.
    - Read and store the values of mcause, mepc, and mstatus.
    - Update the timer compare value to schedule the next interrupt.
6. Return from the interrupt handler using mret.
7. The main program should continue execution after the interrupt.

Observations Required

- mcause value indicating a machine timer interrupt
- mepc value at the time of interrupt
- Changes in mstatus during interrupt entry and exit

Problem 3: Privilege Enforcement and Illegal Instruction Trap (50 points)

Write a RISC-V assembly program that demonstrates **privilege enforcement** by executing a **privileged instruction in User mode**.

Program Requirements

1. Start execution in **Machine mode**.
2. Set the mtvec register to point to a custom machine-mode trap handler.
3. Modify mstatus and mepc to switch to **User mode** using mret.
4. In User mode, attempt to execute a **machine-only instruction** (for example, writing to mstatus).
5. This should cause an **illegal instruction exception**.
6. In the trap handler:
    - Read mcause, mepc, and mtval
    - Identify the illegal instruction exception
    - Increment mepc to skip the offending instruction
7. Return safely to User mode using mret.

Observations Required

- mcause value for illegal instruction exception
- mtval content related to the faulting instruction
- Program continuation without repeated traps

Problem 4: Using mscratch for Context Saving in Trap Handler

(50 points)

Write a RISC-V assembly program to demonstrate the use of the **mscratch register** for temporary storage during trap handling.

Program Requirements

1. Start execution in **Machine mode**.
2. Initialize the mtvec register with the address of a machine-mode trap handler.
3. Store a known value in the mscratch register.
4. Trigger a trap using the ecall instruction.
5. In the trap handler:

    - Use mscratch to save at least two general-purpose registers
    - Read mcause and mepc
    - Restore the saved registers before returning
6. Increment mepc to skip the ecall instruction.
7. Return to the main program using mret.

Observations Required

- Value stored and retrieved from mscratch
- mcause value for environment call exception
- Correct program flow after trap return

Compilation and Execution Instructions

> **Note:**
> When using the proxy kernel (pk), the program executes in User mode.
> To execute in Machine mode, run the program **without pk**.

Compile

riscv64-unknown-elf-gcc -nostartfiles -T link.ld program.s

Run

spike -d a.out

Submission Guidelines

- Submit:
    - Assembly source files (.s)
    - Screenshots showing CSR values during execution
- Clearly label screenshots for each problem
- Comment your code properly

****